

Marketplace Revenue Leakage & Gap Analysis using SQL

Identifying Hidden Profit Loss in E-commerce
Marketplace

Presented by: Vidhya M

Problem Statement & Objective

Problem

- Online marketplace sales are growing
- But profits are declining
- Hidden costs are silently reducing revenue

Leakage Factors

- Excessive Discounts
- High Return Rates
- Logistics Costs
- Payment Gateway Fees

Objective

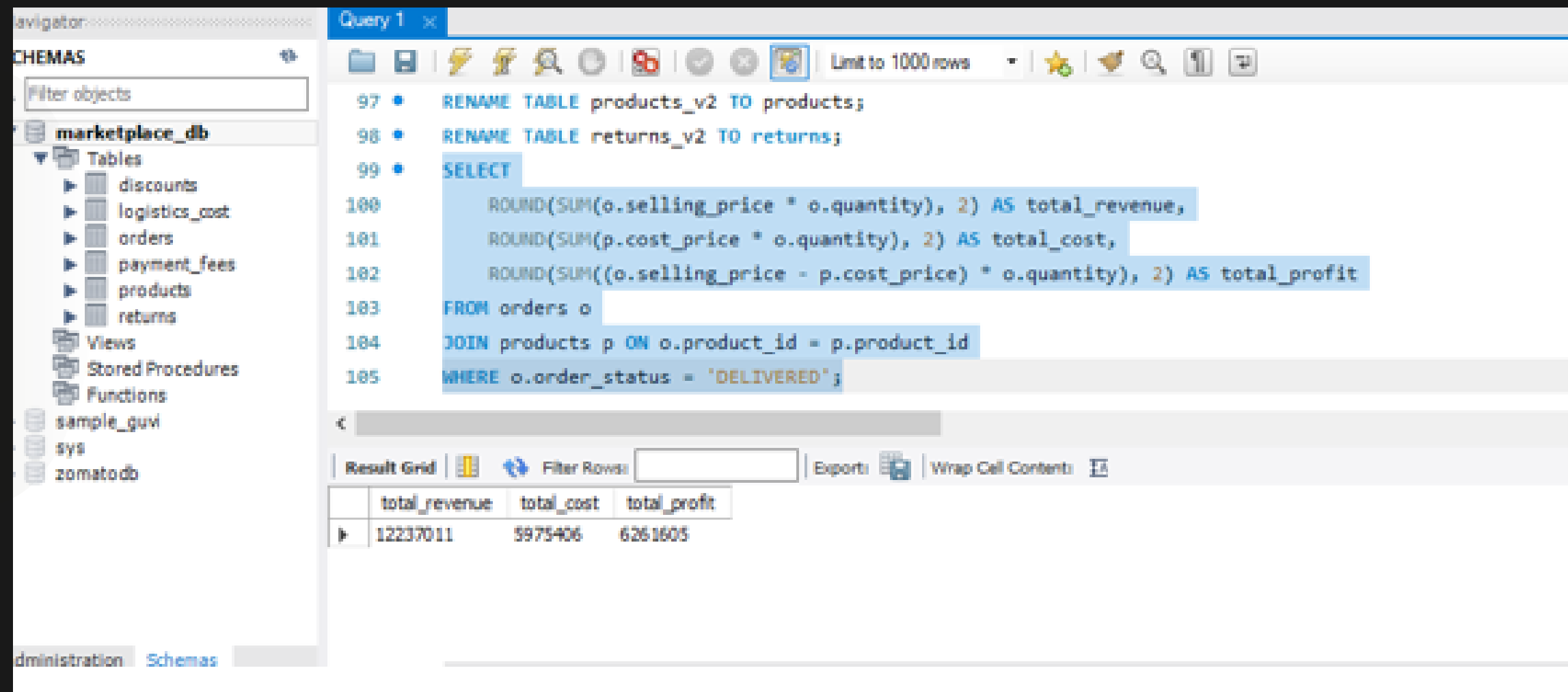
- Design a marketplace database
- Analyze data using SQL
- Find revenue leakage points
- Support better business decisions

Revenue vs Profit Reality

To check if the business is actually making money, not just generating sales

SQL Concepts Used:

- JOIN—Connect orders and products tables
- SUM()—Calculate total revenue, cost, profit
- ROUND() — Format decimal values
- WHERE — Filter only DELIVERED orders



The screenshot shows a SQL IDE interface with a database navigator on the left and a query editor on the right. The database navigator shows a schema named 'marketplace_db' with tables: discounts, logistics_cost, orders, payment_fees, products, and returns. The query editor shows a query named 'Query 1' with the following SQL code:

```
97 • RENAME TABLE products_v2 TO products;
98 • RENAME TABLE returns_v2 TO returns;
99 • SELECT
100     ROUND(SUM(o.selling_price * o.quantity), 2) AS total_revenue,
101     ROUND(SUM(p.cost_price * o.quantity), 2) AS total_cost,
102     ROUND(SUM((o.selling_price - p.cost_price) * o.quantity), 2) AS total_profit
103 FROM orders o
104 JOIN products p ON o.product_id = p.product_id
105 WHERE o.order_status = 'DELIVERED';
```

The query results are displayed in a table with the following data:

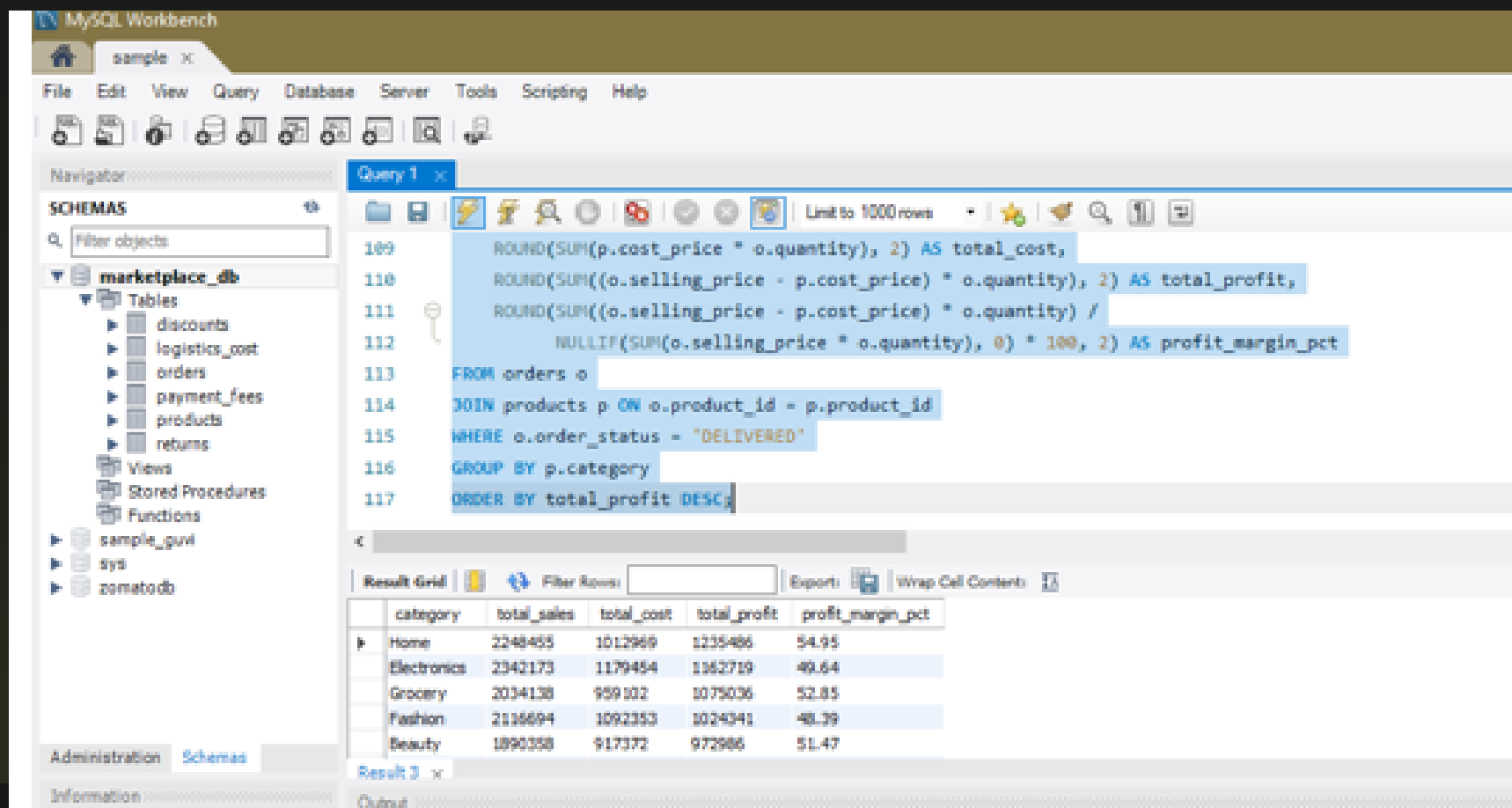
total_revenue	total_cost	total_profit
12237011	5975406	6261605

Category Performance & Loss-Making Products

To find which categories make profit and which ones need improvement

SQL Concepts Used:

- GROUP BY — Group data by category
- SUM() — Calculate sales, cost, profit
- ROUND() — Format decimal values
- ORDER BY — Sort by highest profit



The screenshot shows the MySQL Workbench interface. The 'Query 1' window contains a SQL query that calculates total sales, total cost, total profit, and profit margin percentage for each category. The 'Result Grid' shows the output of this query, sorted by total profit in descending order.

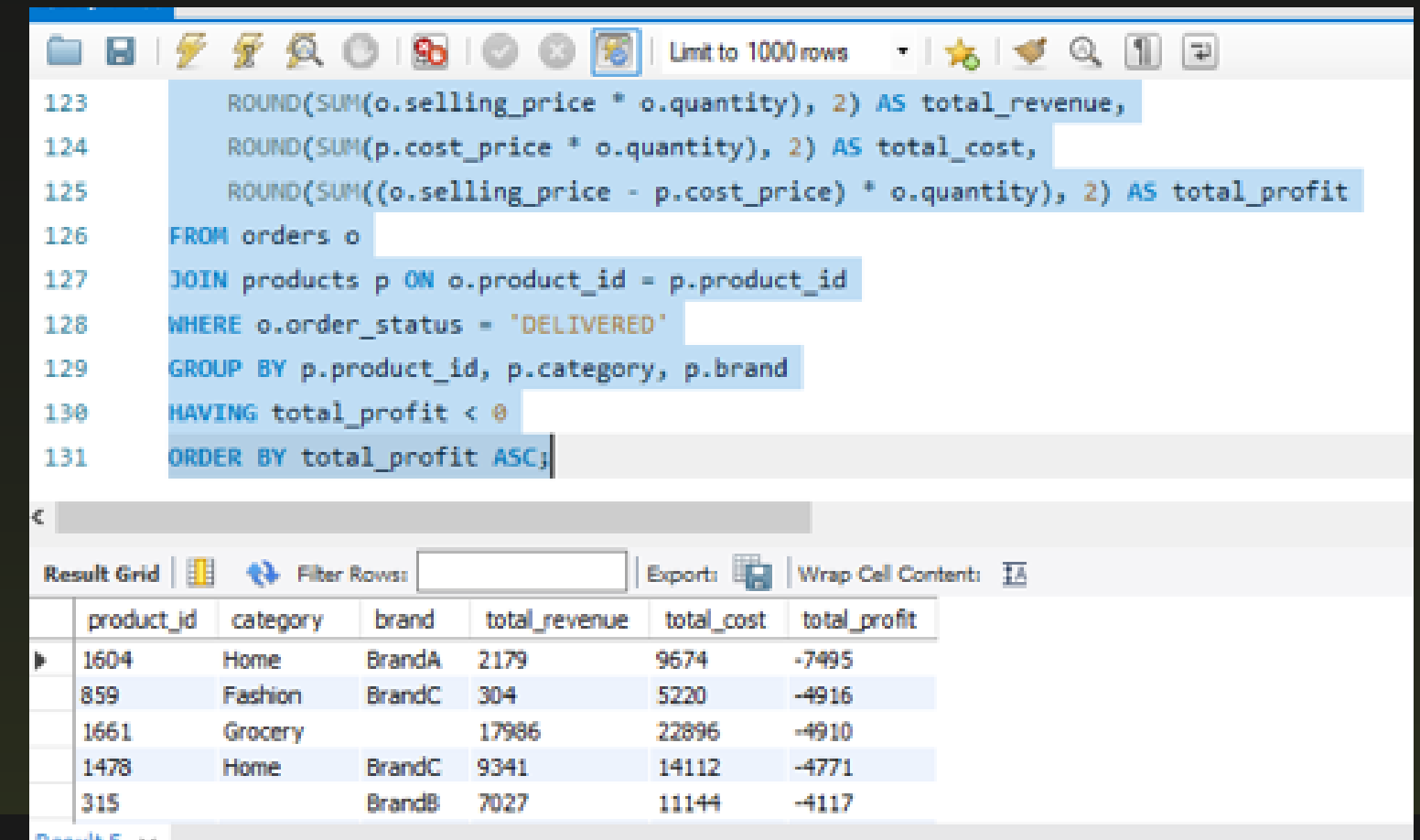
```
109 ROUND(SUM(p.cost_price * o.quantity), 2) AS total_cost,
110 ROUND(SUM((o.selling_price - p.cost_price) * o.quantity), 2) AS total_profit,
111 ROUND(SUM((o.selling_price - p.cost_price) * o.quantity) /
112     NULLIF(SUM(o.selling_price * o.quantity), 0) * 100, 2) AS profit_margin_pct
113 FROM orders o
114 JOIN products p ON o.product_id = p.product_id
115 WHERE o.order_status = 'DELIVERED'
116 GROUP BY p.category
117 ORDER BY total_profit DESC;
```

category	total_sales	total_cost	total_profit	profit_margin_pct
Home	2248455	1012969	1235486	54.95
Electronics	2342173	1179484	1162719	49.64
Grocery	2034138	959102	1075036	52.85
Fashion	2116694	1092353	1024341	48.39
Beauty	1890358	917372	972986	51.47

To identify products that are silently losing money

SQL Concepts Used:

- GROUP BY — Group by product
- SUM() — Calculate total profit per product
- HAVING — Filter only negative profit products
- ORDER BY — Sort by highest loss first



The screenshot shows the MySQL Workbench interface. The 'Query 1' window contains a SQL query that identifies products with negative total profit. The 'Result Grid' shows the output of this query, sorted by total profit in ascending order (highest loss first).

```
123 ROUND(SUM(o.selling_price * o.quantity), 2) AS total_revenue,
124 ROUND(SUM(p.cost_price * o.quantity), 2) AS total_cost,
125 ROUND(SUM((o.selling_price - p.cost_price) * o.quantity), 2) AS total_profit
126 FROM orders o
127 JOIN products p ON o.product_id = p.product_id
128 WHERE o.order_status = 'DELIVERED'
129 GROUP BY p.product_id, p.category, p.brand
130 HAVING total_profit < 0
131 ORDER BY total_profit ASC;
```

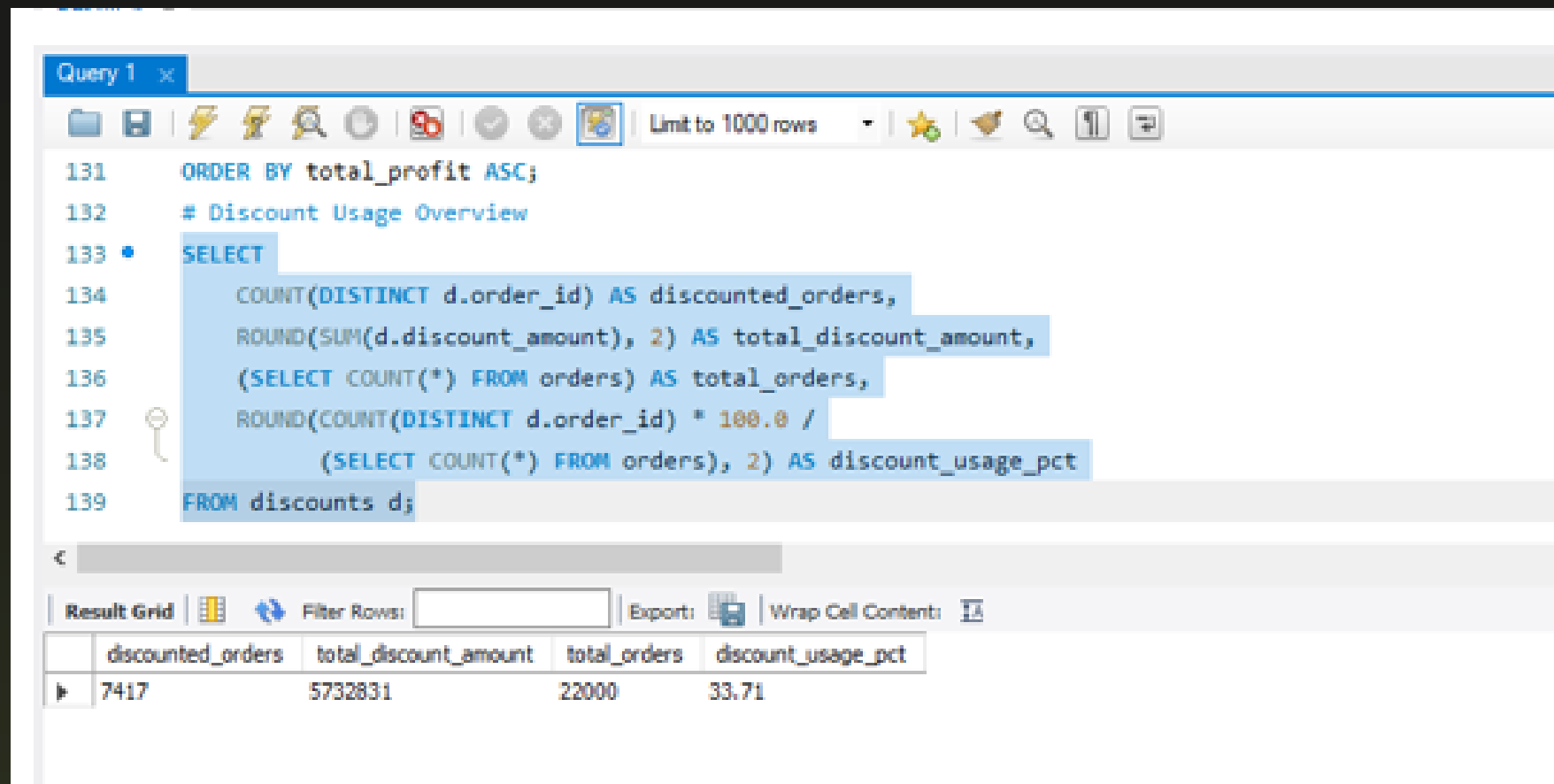
product_id	category	brand	total_revenue	total_cost	total_profit
1604	Home	BrandA	2179	9674	-7495
859	Fashion	BrandC	304	5220	-4916
1661	Grocery		17986	22896	-4910
1478	Home	BrandC	9341	14112	-4771
315		BrandB	7027	11144	-4117

Discount Usage & Payment Method Popularity

To understand how aggressively discounts are used and how much revenue is lost

SQL Concepts Used:

- COUNT() — Count discounted orders
- SUM() — Total discount amount
- SUBQUERY — Get total orders inside query
- ROUND() — Format percentage values



The screenshot shows a SQL IDE window titled 'Query 1'. The query is as follows:

```
131 ORDER BY total_profit ASC;
132 # Discount Usage Overview
133 SELECT
134     COUNT(DISTINCT d.order_id) AS discounted_orders,
135     ROUND(SUM(d.discount_amount), 2) AS total_discount_amount,
136     (SELECT COUNT(*) FROM orders) AS total_orders,
137     ROUND(COUNT(DISTINCT d.order_id) * 100.0 /
138           (SELECT COUNT(*) FROM orders), 2) AS discount_usage_pct
139 FROM discounts d;
```

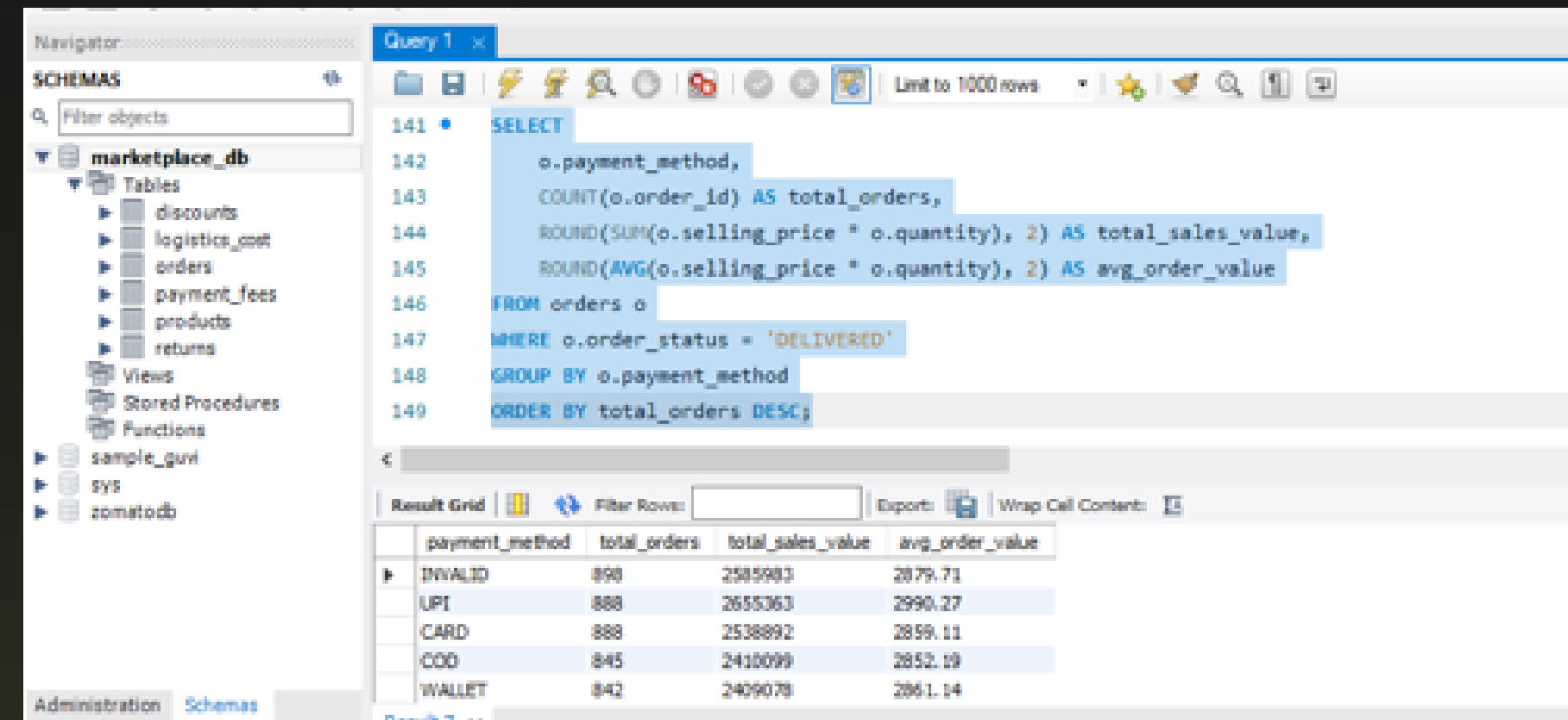
The result grid shows the following data:

discounted_orders	total_discount_amount	total_orders	discount_usage_pct
7417	5732831	22000	33.71

To understand which payment methods customers prefer most

SQL Concepts Used:

- GROUP BY — Group by payment method
- COUNT() — Count orders per method
- SUM() — Total sales per method
- AVG() — Average order value per method



The screenshot shows a SQL IDE window titled 'Query 1'. The query is as follows:

```
141 SELECT
142     o.payment_method,
143     COUNT(o.order_id) AS total_orders,
144     ROUND(SUM(o.selling_price * o.quantity), 2) AS total_sales_value,
145     ROUND(AVG(o.selling_price * o.quantity), 2) AS avg_order_value
146 FROM orders o
147 WHERE o.order_status = 'DELIVERED'
148 GROUP BY o.payment_method
149 ORDER BY total_orders DESC;
```

The result grid shows the following data:

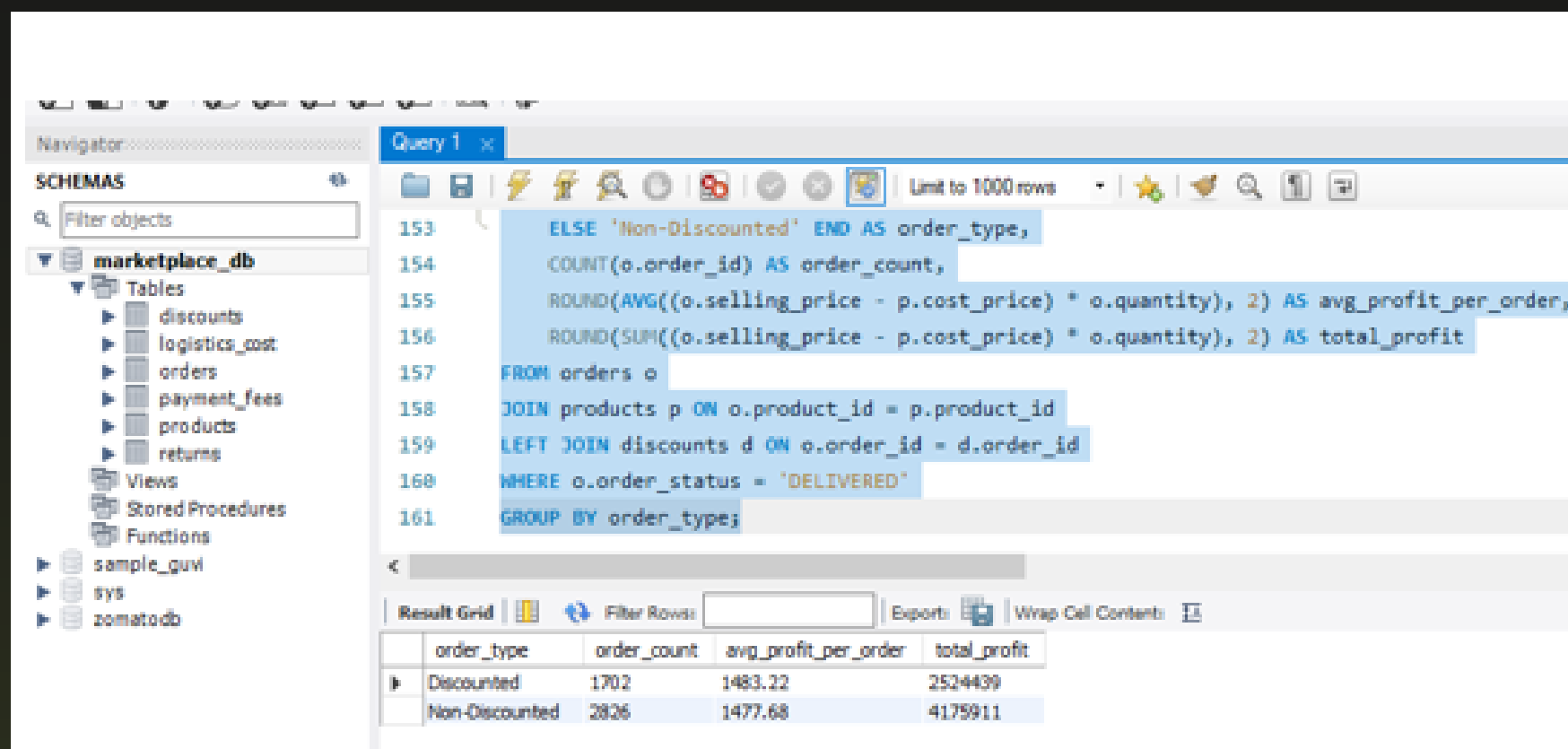
payment_method	total_orders	total_sales_value	avg_order_value
INVALID	898	2585983	2879.71
UPI	888	2655363	2990.27
CARD	888	2538892	2859.11
COD	845	2410099	2852.19
WALLET	842	2409078	2861.14

Discount Effectiveness & Return Revenue Loss

To check if discounts actually improve profit or only increase order volume

SQL Concepts Used:

- LEFT JOIN—Include orders with and without discount
- CASE WHEN—Label discounted vs non-discounted orders
- AVG() — Compare average profit per order type
- GROUP BY — Group by order type



The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
153 ELSE 'Non-Discounted' END AS order_type,  
154 COUNT(o.order_id) AS order_count,  
155 ROUND(AVG((o.selling_price - p.cost_price) * o.quantity), 2) AS avg_profit_per_order,  
156 ROUND(SUM((o.selling_price - p.cost_price) * o.quantity), 2) AS total_profit  
157 FROM orders o  
158 JOIN products p ON o.product_id = p.product_id  
159 LEFT JOIN discounts d ON o.order_id = d.order_id  
160 WHERE o.order_status = 'DELIVERED'  
161 GROUP BY order_type;
```

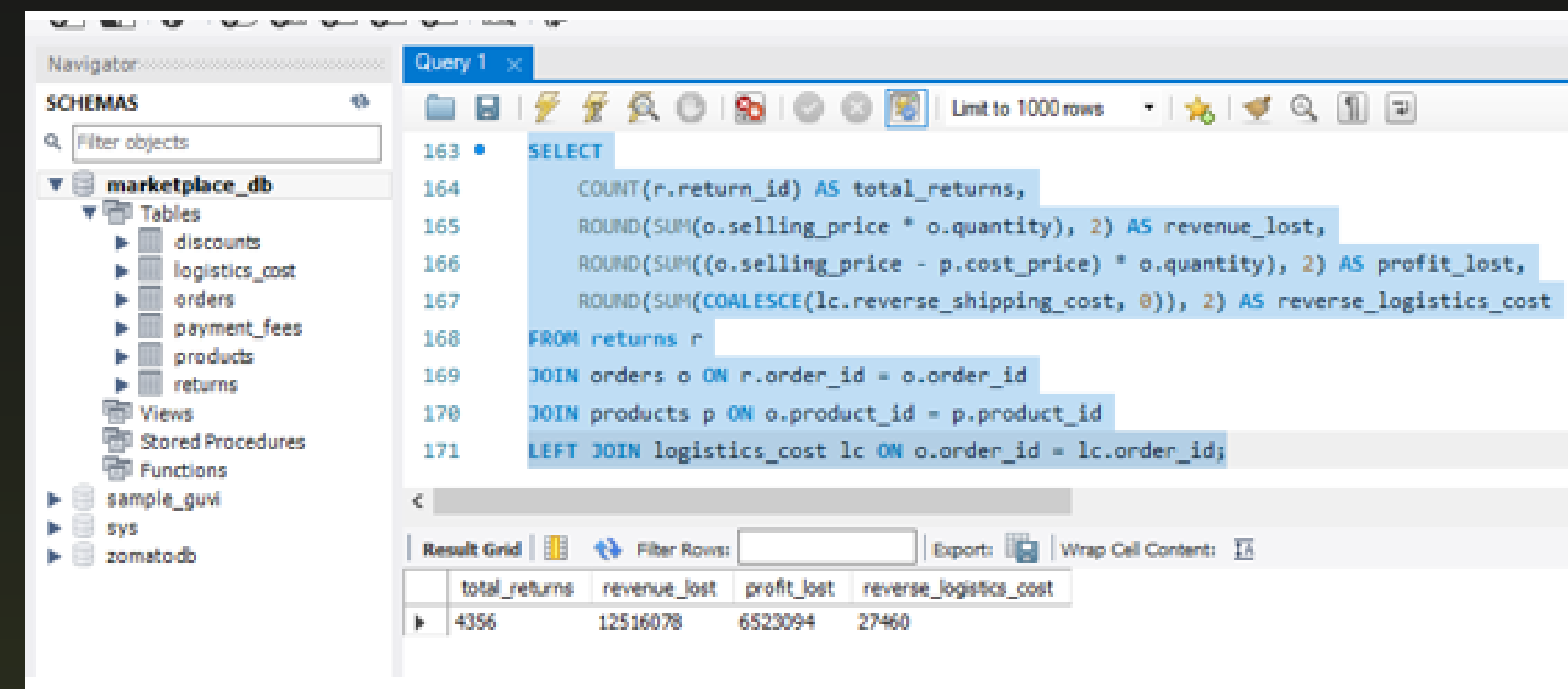
The result grid shows the following data:

order_type	order_count	avg_profit_per_order	total_profit
Discounted	1702	1483.22	2524439
Non-Discounted	2826	1477.68	4175911

To measure how much revenue and profit is lost due to customer returns

SQL Concepts Used:

- JOIN — Connect returns, orders, products
- LEFT JOIN—Include logistics cost if available
- SUM() — Calculate total revenue and profit lost
- COALESCE() — Handle NULL shipping cost values



The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
163 SELECT  
164 COUNT(r.return_id) AS total_returns,  
165 ROUND(SUM(o.selling_price * o.quantity), 2) AS revenue_lost,  
166 ROUND(SUM((o.selling_price - p.cost_price) * o.quantity), 2) AS profit_lost,  
167 ROUND(SUM(COALESCE(lc.reverse_shipping_cost, 0)), 2) AS reverse_logistics_cost  
168 FROM returns r  
169 JOIN orders o ON r.order_id = o.order_id  
170 JOIN products p ON o.product_id = p.product_id  
171 LEFT JOIN logistics_cost lc ON o.order_id = lc.order_id;
```

The result grid shows the following data:

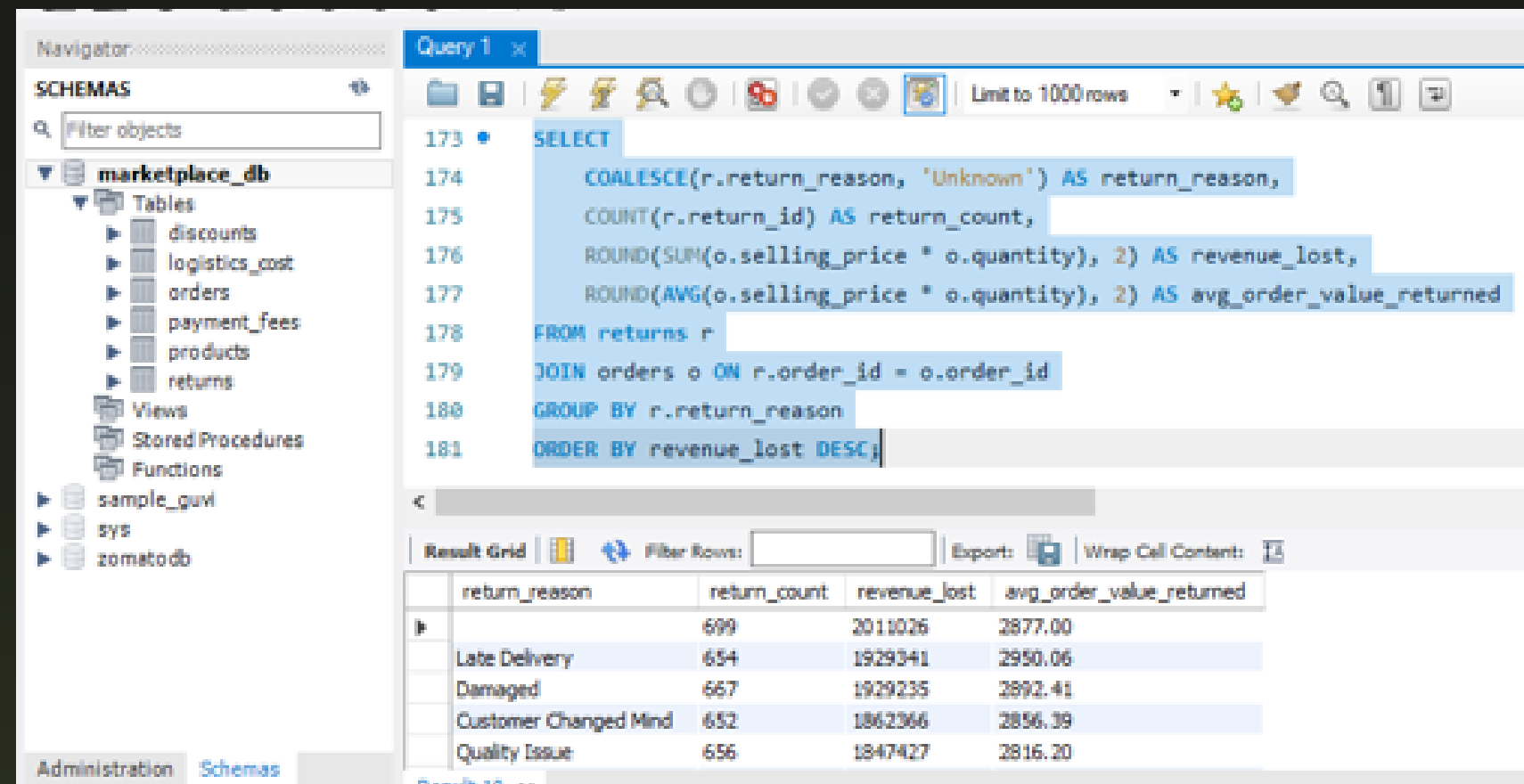
total_returns	revenue_lost	profit_lost	reverse_logistics_cost
4356	12516078	6523094	27460

Return Reasons & Logistics Cost Burden

To understand why customers return products and which reason causes highest revenue loss

SQL Concepts Used:

- COALESCE() — Replace NULL reasons with Unknown
- GROUP BY — Group by return reason
- SUM() — Total revenue lost per reason
- ORDER BY—Sort by highest revenue loss



The screenshot shows a database query editor with a sidebar on the left displaying the 'marketplace_db' schema. The main area shows a SQL query for 'Query 1' and its results in a grid.

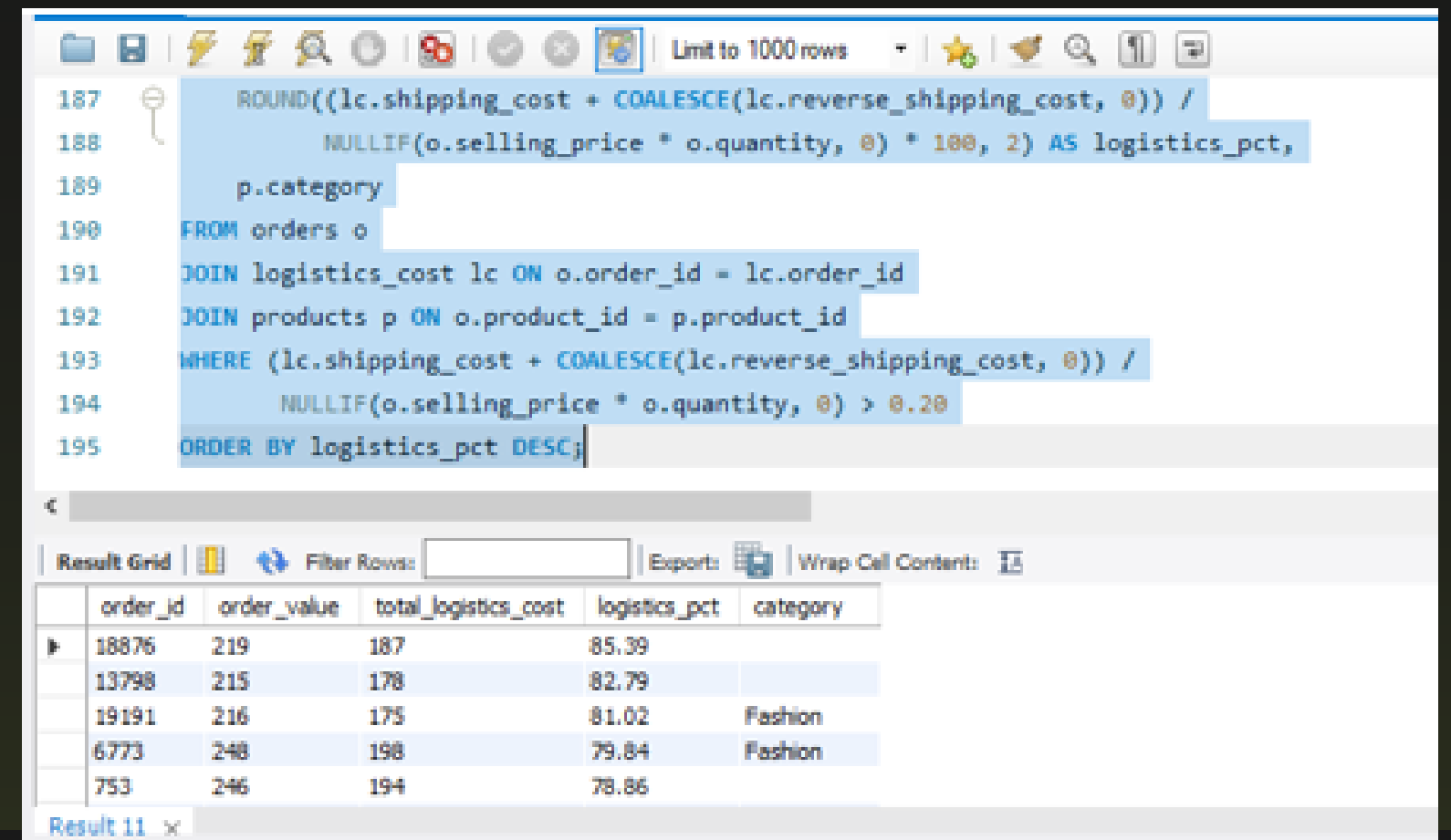
```
SELECT
  COALESCE(r.return_reason, 'Unknown') AS return_reason,
  COUNT(r.return_id) AS return_count,
  ROUND(SUM(o.selling_price * o.quantity), 2) AS revenue_lost,
  ROUND(AVG(o.selling_price * o.quantity), 2) AS avg_order_value_returned
FROM returns r
JOIN orders o ON r.order_id = o.order_id
GROUP BY r.return_reason
ORDER BY revenue_lost DESC;
```

return_reason	return_count	revenue_lost	avg_order_value_returned
	699	2011026	2877.00
Late Delivery	654	1929341	2950.06
Damaged	667	1929235	2892.41
Customer Changed Mind	652	1862366	2856.39
Quality Issue	656	1847427	2816.20

To find orders where delivery cost is too high compared to order value

SQL Concepts Used:

- COALESCE() — Handle NULL reverse shipping values
- NULLIF() — Avoid divide by zero errors
- WHERE — Filter orders where logistics > 20%
- ORDER BY — Sort by highest logistics percentage



The screenshot shows a database query editor with a sidebar on the left displaying the 'marketplace_db' schema. The main area shows a SQL query for 'Query 1' and its results in a grid.

```
ROUND((lc.shipping_cost + COALESCE(lc.reverse_shipping_cost, 0)) /
  NULLIF(o.selling_price * o.quantity, 0) * 100, 2) AS logistics_pct,
  p.category
FROM orders o
JOIN logistics_cost lc ON o.order_id = lc.order_id
JOIN products p ON o.product_id = p.product_id
WHERE (lc.shipping_cost + COALESCE(lc.reverse_shipping_cost, 0)) /
  NULLIF(o.selling_price * o.quantity, 0) > 0.20
ORDER BY logistics_pct DESC;
```

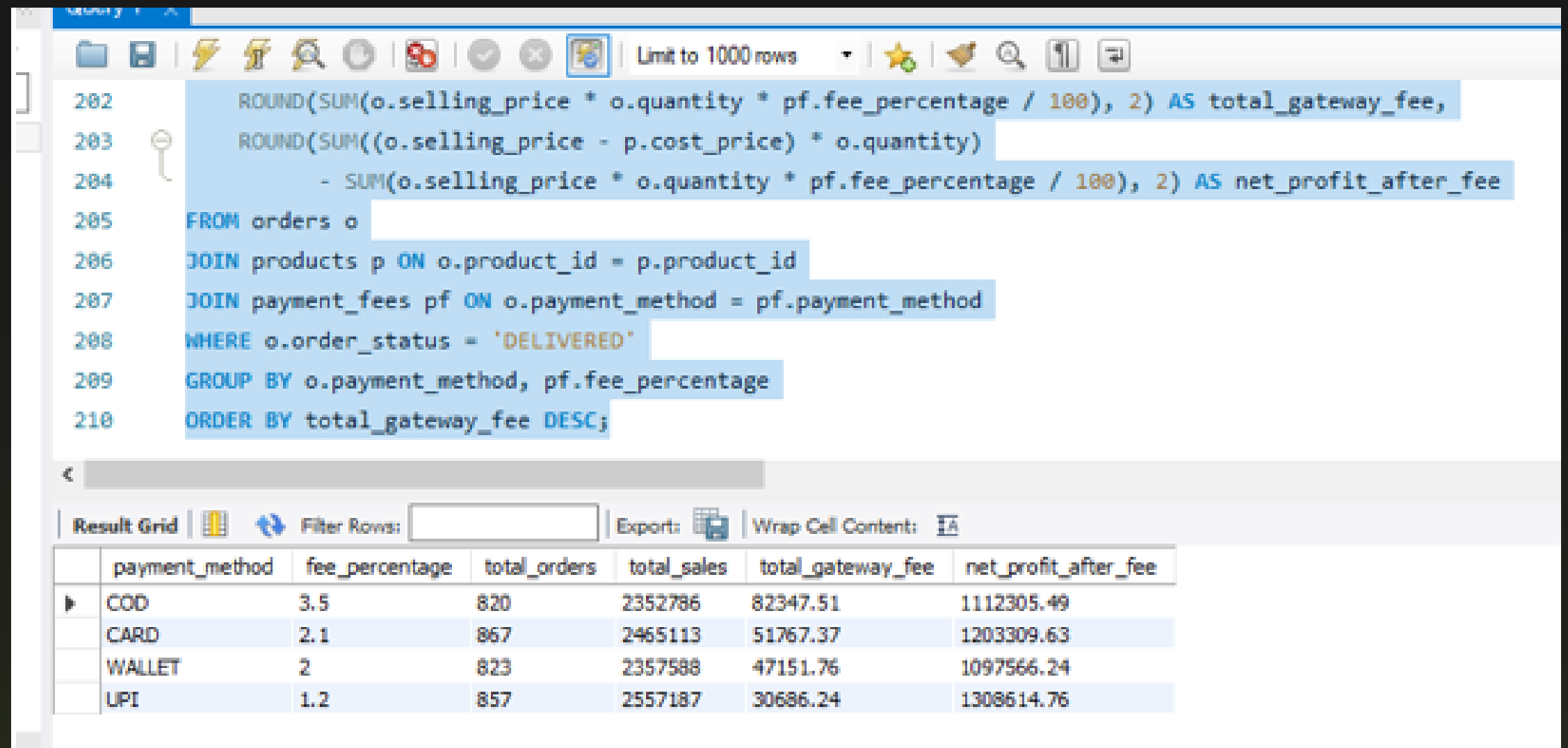
order_id	order_value	total_logistics_cost	logistics_pct	category
18876	219	187	85.39	
13798	215	178	82.79	
19191	216	175	81.02	Fashion
6773	248	198	79.84	Fashion
753	246	194	78.86	

Payment Fee Leakage

To find the hidden cost of each payment method and its real impact on net profit

SQL Concepts Used:

- JOIN—Connect payment_fees and orders tables
- SUM() — Calculate total gateway fees per method
- GROUP BY — Group by payment method
- ROUND() — Format all fee values



```
282 ROUND(SUM(o.selling_price * o.quantity * pf.fee_percentage / 100), 2) AS total_gateway_fee,  
283 ROUND(SUM((o.selling_price - p.cost_price) * o.quantity)  
284 - SUM(o.selling_price * o.quantity * pf.fee_percentage / 100), 2) AS net_profit_after_fee  
285 FROM orders o  
286 JOIN products p ON o.product_id = p.product_id  
287 JOIN payment_fees pf ON o.payment_method = pf.payment_method  
288 WHERE o.order_status = 'DELIVERED'  
289 GROUP BY o.payment_method, pf.fee_percentage  
210 ORDER BY total_gateway_fee DESC;
```

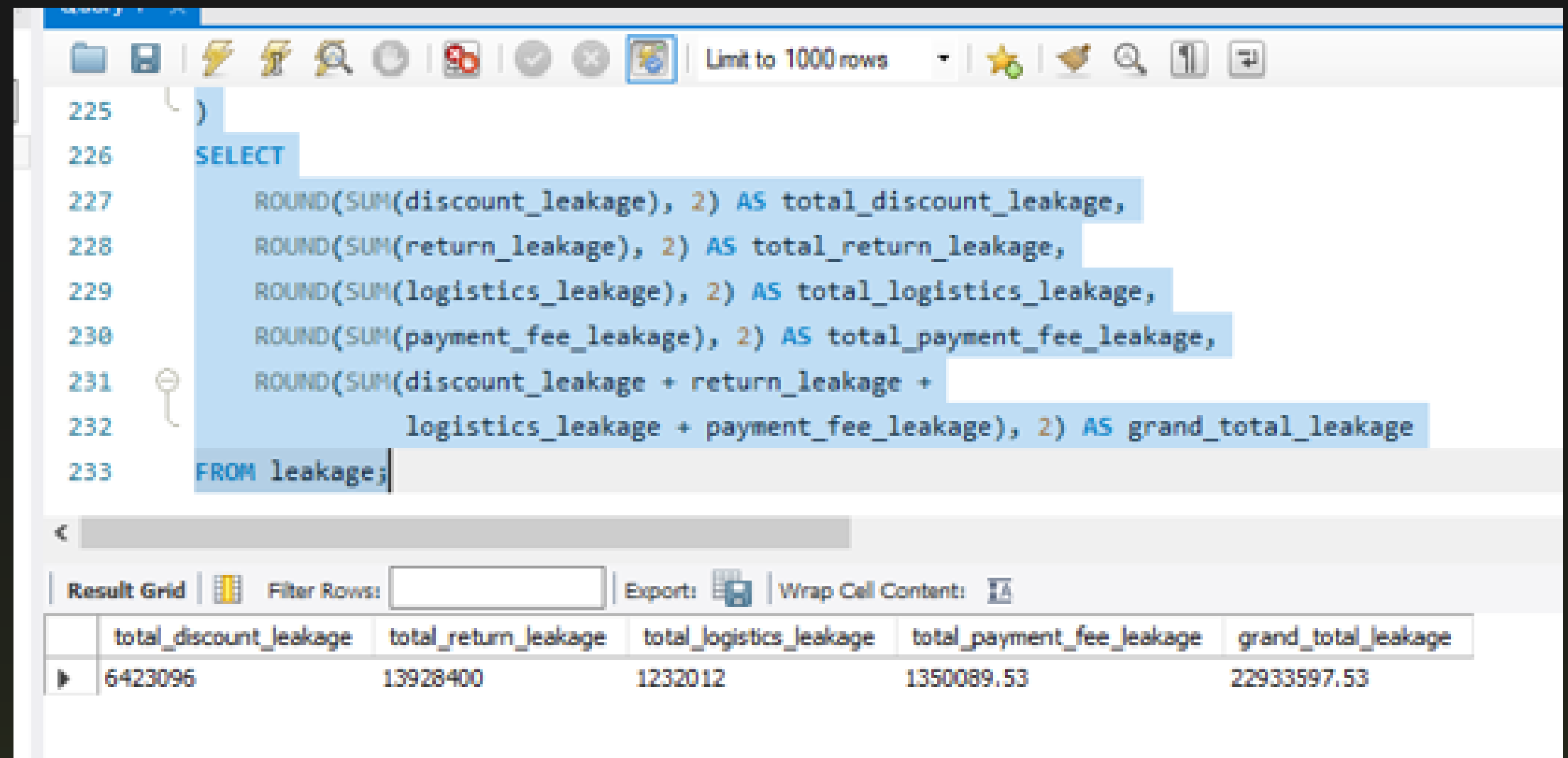
	payment_method	fee_percentage	total_orders	total_sales	total_gateway_fee	net_profit_after_fee
▶	COD	3.5	820	2352786	82347.51	1112305.49
	CARD	2.1	867	2465113	51767.37	1203309.63
	WALLET	2	823	2357588	47151.76	1097566.24
	UPI	1.2	857	2557187	30686.24	1308614.76

Revenue Leakage Breakdown

To find which factor causes the highest revenue loss across all orders in the marketplace

SQL Concepts Used:

- CTE (WITH clause) — Organize leakage calculation
- CASE WHEN — Identify returned orders
- COALESCE() — Handle NULL values
- Multiple LEFT JOIN — Connect all 5 tables together
- SUM() — Calculate total leakage per factor



The screenshot shows a SQL query editor with a query that calculates the total revenue leakage across five categories: discount, return, logistics, payment fee, and a grand total. The query uses the SUM() function to aggregate leakage values and the ROUND() function to format the results to two decimal places. The query is executed against a table named 'leakagej'.

```
225 )
226 SELECT
227     ROUND(SUM(discount_leakage), 2) AS total_discount_leakage,
228     ROUND(SUM(return_leakage), 2) AS total_return_leakage,
229     ROUND(SUM(logistics_leakage), 2) AS total_logistics_leakage,
230     ROUND(SUM(payment_fee_leakage), 2) AS total_payment_fee_leakage,
231     ROUND(SUM(discount_leakage + return_leakage +
232             logistics_leakage + payment_fee_leakage), 2) AS grand_total_leakage
233 FROM leakagej
```

The result grid below the query shows the following data:

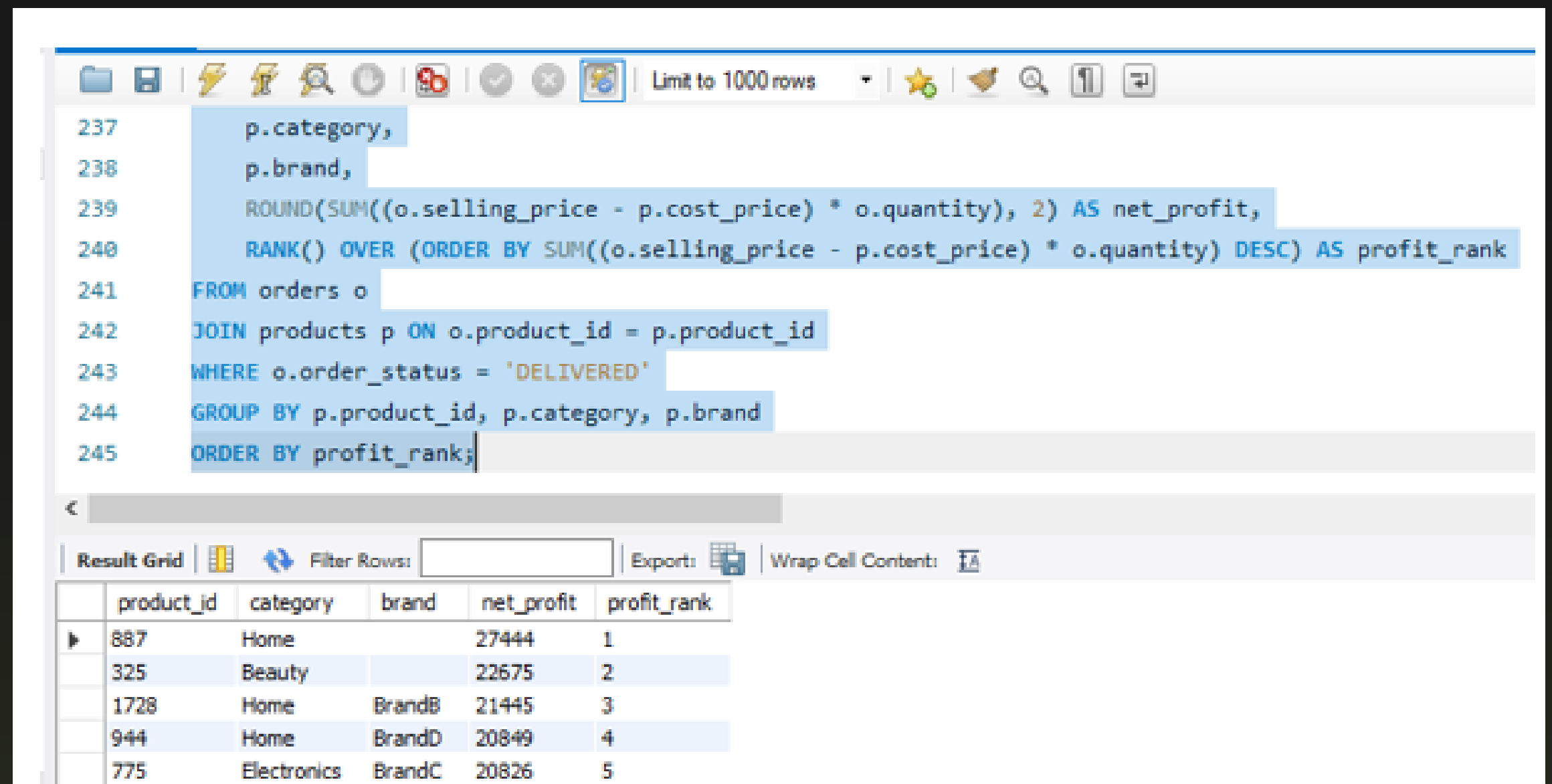
	total_discount_leakage	total_return_leakage	total_logistics_leakage	total_payment_fee_leakage	grand_total_leakage
▶	6423096	13928400	1232012	1350089.53	22933597.53

Product Profit Ranking

To rank all products from highest to lowest profit and identify top and bottom performers

SQL Concepts Used:

- RANK() Window Function — Rank products by net profit
- GROUP BY — Group by product, category, brand
- SUM() — Calculate net profit per product
- ORDER BY — Sort by profit rank



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
237 p.category,  
238 p.brand,  
239 ROUND(SUM((o.selling_price - p.cost_price) * o.quantity), 2) AS net_profit,  
240 RANK() OVER (ORDER BY SUM((o.selling_price - p.cost_price) * o.quantity) DESC) AS profit_rank  
241 FROM orders o  
242 JOIN products p ON o.product_id = p.product_id  
243 WHERE o.order_status = 'DELIVERED'  
244 GROUP BY p.product_id, p.category, p.brand  
245 ORDER BY profit_rank;
```

Below the query editor is a 'Result Grid' showing the output of the query. The grid has columns for product_id, category, brand, net_profit, and profit_rank. The results are sorted by profit rank in descending order.

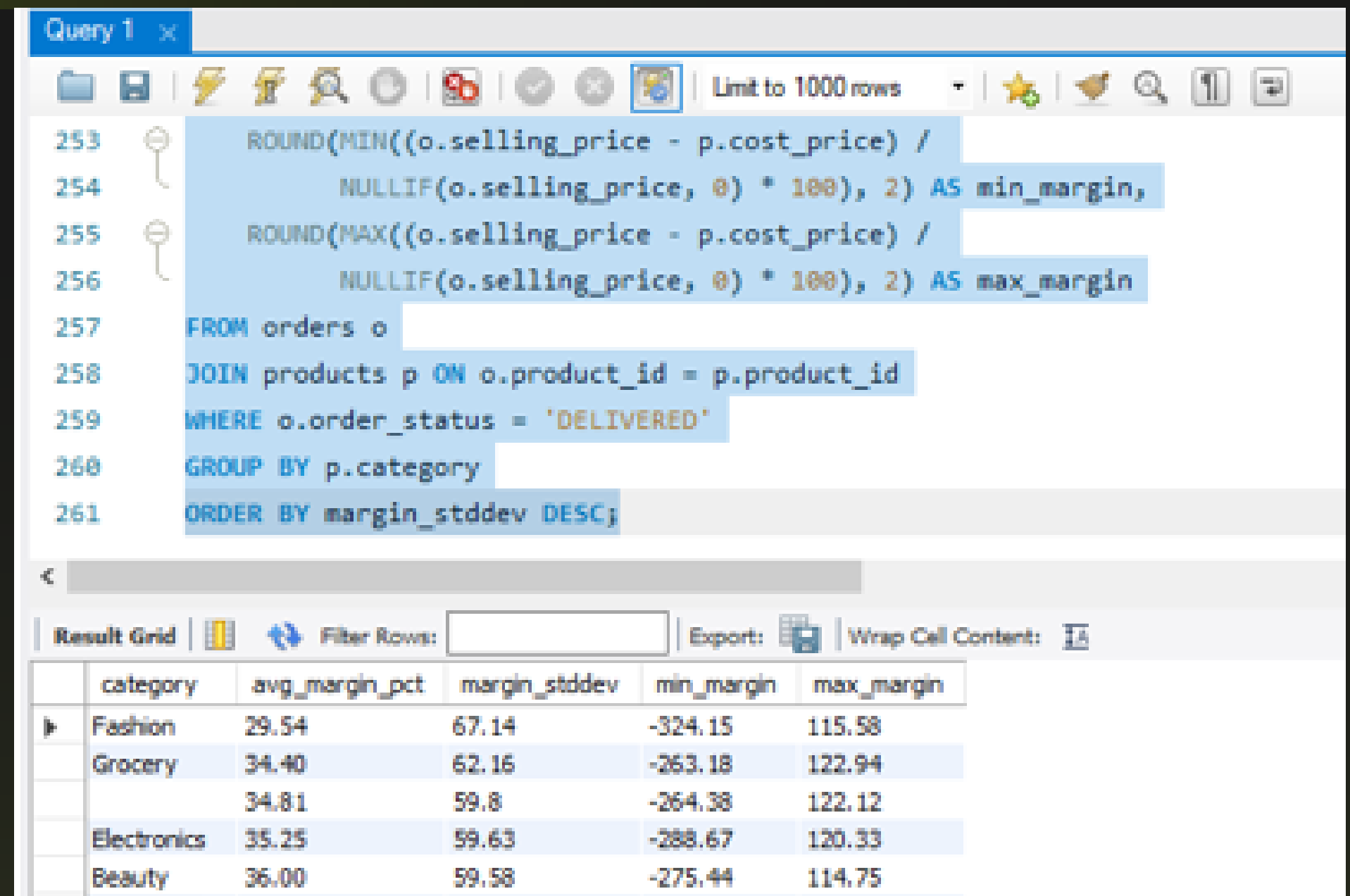
	product_id	category	brand	net_profit	profit_rank
▶	887	Home		27444	1
	325	Beauty		22675	2
	1728	Home	BrandB	21445	3
	944	Home	BrandD	20849	4
	775	Electronics	BrandC	20826	5

Category Margin Stability

To find which categories have unstable profit margins and need better pricing control

SQL Concepts Used:

- STDDEV() — Measure margin variation across orders
- AVG() — Calculate average margin per category
- MIN() — Find lowest margin value
- MAX() — Find highest margin value
- GROUP BY — Group results by category



The screenshot shows a SQL query editor with a query titled "Query 1". The query is as follows:

```
253 ROUND(MIN((o.selling_price - p.cost_price) /  
254     NULLIF(o.selling_price, 0) * 100), 2) AS min_margin,  
255 ROUND(MAX((o.selling_price - p.cost_price) /  
256     NULLIF(o.selling_price, 0) * 100), 2) AS max_margin  
257 FROM orders o  
258 JOIN products p ON o.product_id = p.product_id  
259 WHERE o.order_status = 'DELIVERED'  
260 GROUP BY p.category  
261 ORDER BY margin_stddev DESC;
```

Below the query editor, there is a "Result Grid" showing the results of the query. The grid has 6 columns: category, avg_margin_pct, margin_stddev, min_margin, and max_margin. The results are sorted by margin_stddev in descending order.

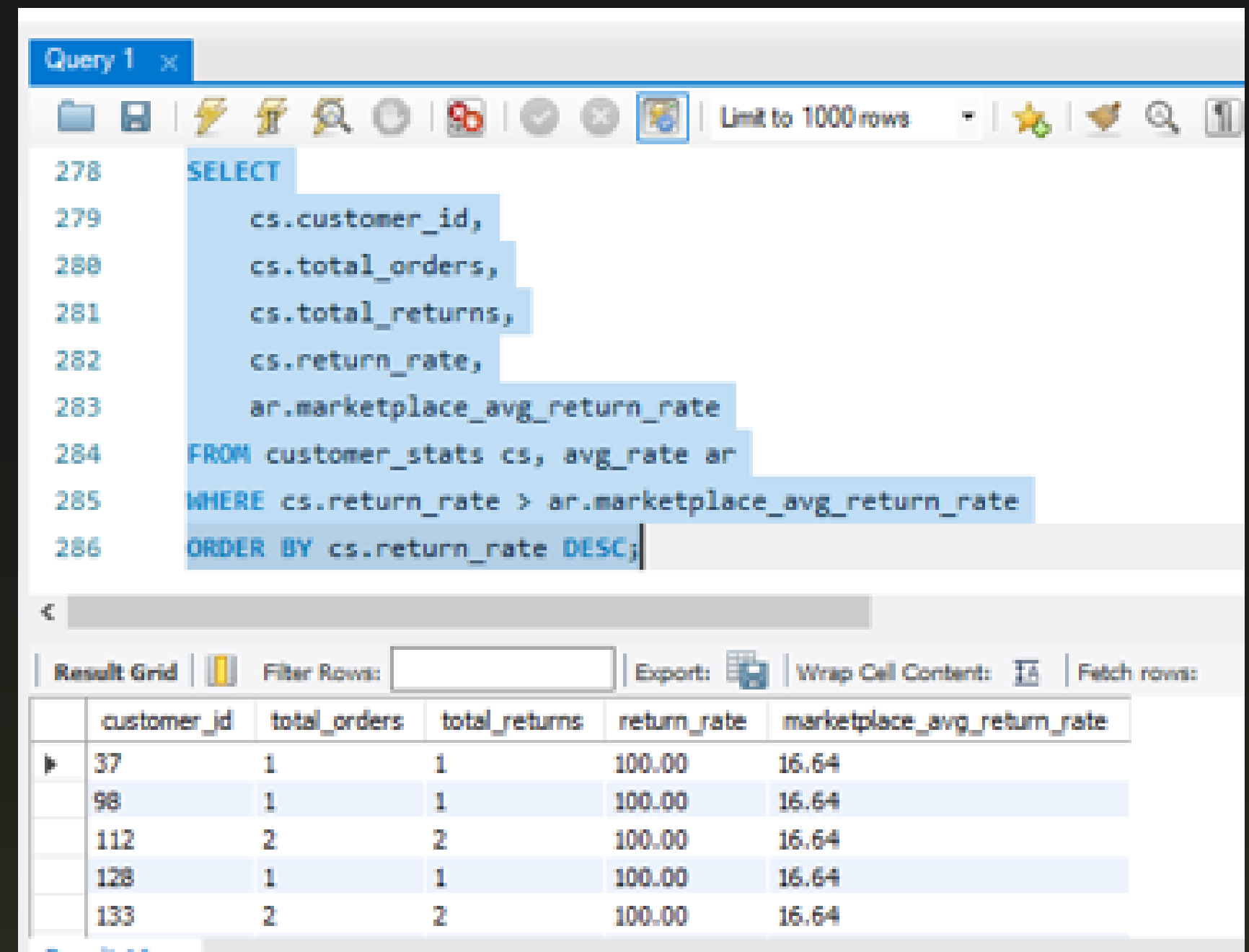
	category	avg_margin_pct	margin_stddev	min_margin	max_margin
▶	Fashion	29.54	67.14	-324.15	115.58
	Grocery	34.40	62.16	-263.18	122.94
		34.81	59.8	-264.38	122.12
	Electronics	35.25	59.63	-288.67	120.33
	Beauty	36.00	59.58	-275.44	114.75

High-Risk Customers

To identify customers who return products too frequently and cause repeated revenue loss to the business

SQL Concepts Used:

- CTE (WITH clause)—Calculate per-customer return stats
- COUNT() — Count orders and returns per customer
- AVG() — Find marketplace average return rate
- LEFT JOIN — Connect orders and returns tables
- WHERE—Filter customers above average rate



```
Query 1 x
SELECT
  cs.customer_id,
  cs.total_orders,
  cs.total_returns,
  cs.return_rate,
  ar.marketplace_avg_return_rate
FROM customer_stats cs, avg_rate ar
WHERE cs.return_rate > ar.marketplace_avg_return_rate
ORDER BY cs.return_rate DESC;
```

	customer_id	total_orders	total_returns	return_rate	marketplace_avg_return_rate
▶	37	1	1	100.00	16.64
	98	1	1	100.00	16.64
	112	2	2	100.00	16.64
	128	1	1	100.00	16.64
	133	2	2	100.00	16.64

Executive Profitability Summary

To get a complete picture of why net profit is lower than expected despite strong total sales

SQL Concepts Used:

- Multiple LEFT JOIN — Connect all 5 tables in one query
- SUM() — Calculate each cost and profit factor
- COALESCE() — Handle NULL cost values
- CASE WHEN — Identify and calculate return losses
- ROUND() — Format all monetary values

```
381      - SUM(COALESCE(lc.shipping_cost + lc.reverse_shipping_cost, 0))
382      - SUM(COALESCE(o.selling_price * o.quantity * pf.fee_percentage / 100, 0))
383      , 2) AS net_profit
384 FROM orders o
385 JOIN products p ON o.product_id = p.product_id
386 LEFT JOIN discounts d ON o.order_id = d.order_id
387 LEFT JOIN returns r ON o.order_id = r.order_id
388 LEFT JOIN logistics_cost lc ON o.order_id = lc.order_id
389 LEFT JOIN payment_fees pf ON o.payment_method = pf.payment_method;
```

	total_sales	gross_profit	total_discounts	total_returns_loss	total_logistics_cost	total_payment_fees	net_profit
	74954710	38618692	6180029	13485146	1192677	1308765.4	16452074.6

Key Recommendations

Reduce Returns

- Improve product quality and packaging
- Set return limits for high-risk customers
- Fix late delivery and wrong item issues
- Save ₹1.28 crore.

Optimize Discounts

- Stop blanket discounts
- Use targeted offers only
- Discounts do not improve profit
- Save ₹64 Lakhs

Cut Logistics & Payment Cost

- Promote UPI — lowest fee 1.2%
- Discourage COD — highest fee 3.5%
- Set minimum order value for free shipping
- Save ₹26 Lakhs

Fix Loss-Making Products

- Reprice or remove 145 loss-making products
- Focus inventory on top-performing products
- Control Fashion category pricing urgently

**Total Savings
₹2.19 Crore**

Recommendations to Increase Profitability

Thank you for
your attention